

Lists, S3, & Python Classes

Lecture 05

Dr. Colin Rundel

R Lists

Lists

Lists are R's other vector type. Unlike atomic vectors they are *heterogeneous* - each element can be any R object (atomic vectors, other lists, functions, ...).

They are a direct analog to Python's `list`.

```
1 list("A", (1:4)/2, list(1L), sum)
```

R

```
[[1]]  
[1] "A"
```

```
[[2]]  
[1] 0.5 1.0 1.5 2.0
```

```
[[3]]  
[[3]][[1]]  
[1] 1
```

```
[[4]]  
function (..., na.rm = FALSE) .Primitive("sum")
```

```
1 ["A", [0.5, 1.0, 1.5, 2.0], [1], sum]
```

py

```
['A', [0.5, 1.0, 1.5, 2.0], [1], <built-in funct
```

List structure

The printed form of a list is verbose, `str()` gives a compact summary of any object's structure and is particularly useful for lists.

```
1 str(c(1, 2))
```

```
num [1:2] 1 2
```

```
1 str(1:100)
```

```
int [1:100] 1 2 3 4 5 6 7 8 9 10 ...
```

```
1 str("A")
```

```
chr "A"
```

```
1 str( list(  
2   "A", c(TRUE, FALSE),  
3   (1:4)/2, list(TRUE, 1),  
4   function(x) x^2  
5 ) )
```

List of 5

```
$ : chr "A"
```

```
$ : logi [1:2] TRUE FALSE
```

```
$ : num [1:4] 0.5 1 1.5 2
```

```
$ :List of 2
```

```
..$ : logi TRUE
```

```
..$ : num 1
```

```
$ :function (x)
```

Nested lists

Lists can contain other lists, so they do not have to be flat.

This makes them a natural way of representing *tree-like* data (e.g. JSON).

```
1 x = list(1, list(2, list(3, 4), 5))  
2 str(x)
```

R

```
List of 2  
 $ : num 1  
 $ :List of 3  
  ..$ : num 2  
  ..$ :List of 2  
   .. ..$ : num 3  
   .. ..$ : num 4  
  ..$ : num 5
```

```
1 x = [1, [2, [3, 4], 5]]  
2 x
```

py

```
[1, [2, [3, 4], 5]]
```

Named lists

Elements of a list (or atomic vector) can be named. Names help avoid magic numbers when accessing elements (more readable code).

A valid name starts with a letter or `.` and contains only letters, numbers, `.`, and `_`.

```
1 x = list(A = 1, B = list(C = 2, D = 3))
2 str(x)
```

```
List of 2
 $ A: num 1
 $ B:List of 2
  ..$ C: num 2
  ..$ D: num 3
```

```
1 names(x)
```

```
[1] "A" "B"
```

Any other name must be surrounded with backticks,

```
1 list("knock knock" = "who's there?")
```

```
$`knock knock`
[1] "who's there?"
```

[vs [[

R has three subsetting operators. * `[` (from last week) typically returns an object of the *same type* * `[[` extracts a *single element* * `$` is shorthand for named lookup with `[[`

```
1 y = list(a = 1, b = 4, c = 7:9)
```

```
1 y[2]
```

```
$b  
[1] 4
```

```
1 str( y[2] )
```

```
List of 1  
 $ b: num 4
```

```
1 y["b"]
```

```
$b  
[1] 4
```

```
1 str( y["b"] )
```

```
List of 1  
 $ b: num 4
```

```
1 y[[2]]
```

```
[1] 4
```

```
1 str( y[[2]] )
```

```
num 4
```

```
1 y[["b"]]
```

```
[1] 4
```

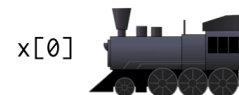
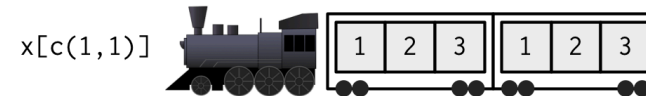
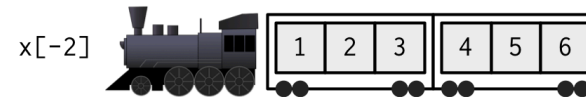
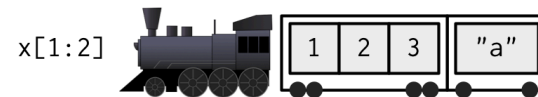
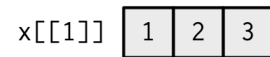
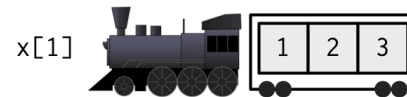
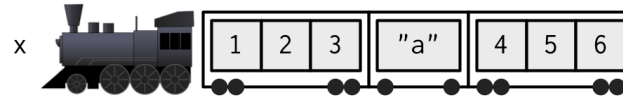
```
1 str( y[["b"]] )
```

```
num 4
```

```
1 y$b
```

```
[1] 4
```

Hadley's analogy



[] details

[] selects by position or by name and only ever returns one element.

```
1 y = list(a = 1, b = 4, c = 7:9)
```

```
1 y[[3]]
```

```
[1] 7 8 9
```

```
1 y[["c"]]
```

```
[1] 7 8 9
```

```
1 y[[4]]
```

```
Error in `y[[4]]`:  
! subscript out of bounds
```

```
1 y[["d"]]
```

```
NULL
```

Vector of length > 1 are interpreted as *recursive* indexing (`y[[c(3, 2)]]` is `y[[3]][[2]]`).

```
1 y[[c(3, 2)]]
```

```
[1] 8
```

```
1 y[[1:2]]
```

```
Error in `y[[1:2]]`:  
! subscript out of bounds
```

\$ subsetting

`$` is a shorthand for `[[` with a *literal* name, so `y$c` is `y[["c"]]`. It only works with lists (and things built on lists, like data frames), and it *partially matches names*.

```
1 y = list(abc = 1, def = 5)
```

R

```
1 y[["abc"]]
```

R

```
[1] 1
```

```
1 y$abc
```

R

```
[1] 1
```

```
1 y$a
```

R

```
[1] 1
```

```
1 y[["a"]]
```

R

```
NULL
```

```
1 x = c(abc = 1, def = 5)
```

R

```
1 x[["abc"]]
```

R

```
[1] 1
```

```
1 x$abc
```

R

```
Error in `x$abc`:  
! $ operator is invalid for atomic vectors
```

A common error is using `$` with a variable that holds a name,

```
1 name = "def"  
2 y[[name]]
```

R

```
[1] 5
```

```
1 y$name
```

R

```
NULL
```

Modifying lists

Assignment with `[[` or `$` replaces an element or adds a new one. Assigning `NULL` removes an element.

```
1 x = list(a = 1, b = 2)
2 x$c = "new"
3 x[["a"]] = 100
4 str(x)
```

List of 3

```
$ a: num 100
$ b: num 2
$ c: chr "new"
```

```
1 x$b = NULL
2 str(x)
```

List of 2

```
$ a: num 100
$ c: chr "new"
```

```
1 x = [1, 2]
2 x.append("new")
3 x[0] = 100
4 x
```

[100, 2, 'new']

```
1 del x[1]
2 x
```

[100, 'new']

Lists and atomic vectors

Combining an atomic vector with a list via `c()` produces a list (the more general type).

`unlist()` goes the other way, flattening a list into an atomic vector with the usual coercion rules.

```
1 str( c(1, list(4, list(6, 7))) )
```

R

List of 3

\$: num 1

\$: num 4

\$:List of 2

..\$: num 6

..\$: num 7

```
1 unlist( list(1:3, list(4:5, 6)) )
```

R

[1] 1 2 3 4 5 6

```
1 unlist( list(1, list(2, list(3, "Hello"))) )
```

R

[1] "1" "2" "3" "Hello"

`unlist()` is frequently paired with `strsplit()`, which returns a list with one character vector per input string,

Exercise 1

Represent the following JSON data as a list in R (we will revisit this in Python next week).

```
1 {  
2   "firstName": "John",  
3   "lastName": "Smith",  
4   "age": 25,  
5   "address":  
6   {  
7     "streetAddress": "21 2nd Street",  
8     "city": "New York",  
9     "state": "NY",  
10    "postalCode": 10021  
11  },  
12  "phoneNumber":  
13  [ {  
14    "type": "home",  
15    "number": "212 555-1239"  
16  },  
17  {  
18    "type": "fax",  
19    "number": "646 555-4567"  
20  } ]  
21 }
```

Once you have the list:

- Extract the fax number using `$` and `[[`.
- Add a `"mobile"` phone number.
- Remove the `age` element.

Attributes

Attributes

Attributes are metadata attached to an R object. Some attributes are special (`names`, `dim`, `dimnames`, `class`, `levels`, ...) because they change how R treats the object, but any attribute can be attached to any object.

Attributes are stored as a *named list* attached to the object, accessed via `attributes()` or `attr()`.

```
1 (x = c(L = 1, M = 2, N = 3))
```

```
L M N  
1 2 3
```

```
1 str( attributes(x) )
```

```
List of 1  
 $ names: chr [1:3] "L" "M" "N"
```

```
1 attr(x, "names")
```

```
[1] "L" "M" "N"
```

```
1 attr(x, "other")
```

```
NULL
```

Setting attributes

The important attributes have helper functions for getting and setting (`names()`, `dim()`, `class()`, `levels()`), which is preferred over `attr()` as the helper functions validate their input.

```
1 names(x) = c("Z", "Y", "X")
2 x
```

```
Z Y X
1 2 3
```

```
1 names(x) = 1:3
2 x
```

```
1 2 3
1 2 3
```

```
1 attributes(x)
```

```
$names
[1] "1" "2" "3"
```

```
1 attr(x, "other") = "anything"
2 x
```

```
1 2 3
1 2 3
attr(,"other")
[1] "anything"
```

```
1 str( attributes(x) )
```

```
List of 2
 $ names: chr [1:3] "1" "2" "3"
 $ other: chr "anything"
```

Note that `names(x) = 1:3` silently coerced the integers to character, since the `names` attribute must be a character vector

Factors

Factors are how R represents categorical data - a variable with a discrete set of possible values (its *levels*).

```
1 (x = factor(c("Sunny", "Cloudy", "Rainy", "Cloudy", "Cloudy")))
```

```
[1] Sunny Cloudy Rainy Cloudy Cloudy  
Levels: Cloudy Rainy Sunny
```

```
1 str(x)
```

```
Factor w/ 3 levels "Cloudy","Rainy",...: 3 1 2 1 1
```

```
1 typeof(x)
```

```
[1] "integer"
```

```
1 mode(x)
```

```
[1] "numeric"
```

```
1 class(x)
```

```
[1] "factor"
```

```
1 levels(x)
```

```
[1] "Cloudy" "Rainy"  "Sunny"
```

Composition

A factor is just an integer vector with two attributes: `levels` and `class`.

```
1 str( attributes(x) )
```

R

```
List of 2
```

```
$ levels: chr [1:3] "Cloudy" "Rainy" "Sunny"  
$ class : chr "factor"
```

```
1 unclass(x)
```

R

```
[1] 3 1 2 1 1  
attr("levels")  
[1] "Cloudy" "Rainy" "Sunny"
```

We can build our own from scratch using `attr()`,

```
1 y = c(3L, 1L, 2L, 1L, 1L)  
2 attr(y, "levels") = c("Cloudy", "Rainy", "Sunny")  
3 attr(y, "class") = "factor"  
4 y
```

R

```
[1] Sunny Cloudy Rainy Cloudy Cloudy  
Levels: Cloudy Rainy Sunny
```

Building objects with `structure()`

Setting attributes one at a time is clunky - `structure()` attaches any number of attributes to an object in a single call and is the standard way to construct objects like this.

```
1 ( y = structure(  
2   c(3L, 1L, 2L, 1L, 1L),  
3   levels = c("Cloudy", "Rainy", "Sunny"),  
4   class = "factor"  
5 ) )
```

```
[1] Sunny Cloudy Rainy Cloudy Cloudy  
Levels: Cloudy Rainy Sunny
```

```
1 class(y)
```

```
[1] "factor"
```

```
1 is.factor(y)
```

```
[1] TRUE
```

```
1 identical(x, y)
```

```
[1] TRUE
```

Factors are integer vectors?

Knowing that factors are stored as integers explains some of their more surprising behaviors,

```
1 x + 1
```

R

Warning in Ops.factor(x, 1): '+' not meaningful for factors

```
[1] NA NA NA NA NA
```

```
1 is.integer(x)
```

R

```
[1] FALSE
```

```
1 is.numeric(x)
```

R

```
[1] FALSE
```

```
1 as.integer(x)
```

R

```
[1] 3 1 2 1 1
```

```
1 as.character(x)
```

R

```
[1] "Sunny" "Cloudy" "Rainy" "Cloudy" "C
```

```
1 as.logical(x)
```

R

```
[1] NA NA NA NA NA
```

S3 Object System

class

The `class` attribute adds a layer on top of R's type hierarchy - previously we saw `typeof()` and `mode()`, `class()` is what determines how an object is *treated* by functions like `print()`.

value	typeof()	mode()	class()
TRUE	logical	logical	logical
1	double	numeric	numeric
1L	integer	numeric	integer
"A"	character	character	character
NULL	NULL	NULL	NULL
list(1, "A")	list	list	list
factor("A")	integer	numeric	factor
matrix(1:4, 2)	integer	numeric	matrix, array
function(x) x^2	closure	function	function
sum	builtin	function	function

S3 class specialization

```
1 x = c("A", "B", "A", "C")
```

R

```
1 print( x )
```

R

```
[1] "A" "B" "A" "C"
```

```
1 print( factor(x) )
```

R

```
[1] A B A C  
Levels: A B C
```

```
1 print( unclass( factor(x) ) )
```

R

```
[1] 1 2 1 3  
attr("levels")  
[1] "A" "B" "C"
```

```
1 print.default( factor(x) )
```

R

```
[1] 1 2 1 3
```

What's up with `print`?

```
1 print
```

R

```
function (x, ...)  
UseMethod("print")  
<bytecode: 0x75b1838bd8>  
<environment: namespace:base>
```

`print` does no printing itself - `UseMethod()` looks at the class of `x` and calls the matching *method*. For a factor that is `print.factor()`, for everything without a more specific method `print.default()` is used.

Generics are everywhere

Many of the base R functions you use regularly are S3 generics whose only job is to dispatch on class,

```
1 mean
```

```
R
```

```
function (x, ...)  
  UseMethod("mean")  
<bytecode: 0x75b000e4a0>  
<environment: namespace:base>
```

```
1 plot
```

```
R
```

```
function (x, y, ...)  
  UseMethod("plot")  
<bytecode: 0x75aec3aac0>  
<environment: namespace:base>
```

```
1 summary
```

```
R
```

```
function (object, ...)  
  UseMethod("summary")  
<bytecode: 0x75aa24d700>  
<environment: namespace:base>
```

```
1 t.test
```

```
R
```

```
function (x, ...)  
  UseMethod("t.test")  
<bytecode: 0x75b11b9dc8>  
<environment: namespace:stats>
```

Other generics, such as `sum`, dispatch without an explicit call to `UseMethod()`:

```
1 sum
```

```
R
```

```
function (... , na.rm = FALSE) .Primitive("sum")
```

`sum` is a primitive implemented in C. It still performs S3 dispatch internally, including to methods such as `Summary.class`,

What is S3?

S3 is R's first and simplest OO system. It is the only OO system used in the base and stats packages, and it's the most commonly used system in CRAN packages. S3 is informal and ad hoc, but it has a certain elegance in its minimalism: you can't take away any part of it and still have a useful OO system.

- Hadley Wickham, Advanced R

Dispatch

S3 uses a very simple dispatch mechanism - a generic function calls `UseMethod("func")`, which looks for a function named `<func>.<class>` for each class of the first argument in turn, and finally `<func>.default`.

`methods()` lists the methods available for a generic, or all the methods available for a class,

```
1 methods("summary")
```

```
[1] summary,ANY-method          summary,diagonalMatrix-method
[3] summary,sparseMatrix-method summary.aov
[5] summary.aovlist*            summary.aspell*
[7] summary.check_packages_in_dir* summary.connection
[9] summary.data.frame          summary.Date
[11] summary.default             summary.difftime
[13] summary.ecdf*               summary.factor
[15] summary.free1way*           summary.glm
[17] summary.infl*               summary.lm
[19] summary.loess*              summary.manova
[21] summary.matrix              summary.mlm*
[23] summary.nls*                summary.packageStatus*
[25] summary.pandas.core.frame.DataFrame* summary.pandas.core.series.Series*
[27] summary.pandas.DataFrame*   summary.pandas.Series*
[29] summary.POSIXct             summary.POSIXlt
[31] summary.ppr*                summary.prcomp*
[33] summary.princomp*           summary.proc_time
```

R

[35]	summary.python.builtin.object*	summary.rlang_error*
[37]	summary.rlang_message*	summary.rlang_trace*
[39]	summary.rlang_warning*	summary.rlang:::list_of_conditions*
[41]	summary.shingle*	summary.srcfile
[43]	summary.srcfile	summary.stepfun
[45]	summary.table*	summary.table

```
1 methods(class = "factor")
```

[1]	[	[[	[[<-	[<-	all.equal
[6]	Arith	as.character	as.data.frame	as.Date	as.list
[11]	as.logical	as.POSIXlt	as.vector	c	cbind2
[16]	coerce	Compare	droplevels	format	free1way
[21]	initialize	is.na<-	kronecker	length<-	levels<-
[26]	Logic	Math	Ops	plot	print
[31]	rbind2	relevel	relist	rep	show
[36]	slotsFromS3	summary	Summary	xtfrm	

see '?methods' for accessing help and source code

Adding methods

Because dispatch is by *name*, adding a method for a new class is just defining a function with the proper name - no registration is needed.

```
1 ( x = structure(  
2   c(1, 2, 3),  
3   class = "class_A") )
```

```
[1] 1 2 3  
attr(,"class")  
[1] "class_A"
```

```
1 print.class_A = function(x, ...) {  
2   cat("(Class A) ")  
3   print.default(unclass(x))  
4 }
```

```
1 print(x)
```

```
(Class A) [1] 1 2 3
```

```
1 class(x) = "class_B"  
2 print(x)
```

```
(Class B) [1] 1 2 3
```

```
1 ( y = structure(  
2   c(6, 5, 4),  
3   class = "class_B") )
```

```
[1] 6 5 4  
attr(,"class")  
[1] "class_B"
```

```
1 print.class_B = function(x, ...) {  
2   cat("(Class B) ")  
3   print.default(unclass(x))  
4 }
```

```
1 print(y)
```

```
(Class B) [1] 6 5 4
```

```
1 class(y) = "class_A"  
2 print(y)
```

```
(Class A) [1] 6 5 4
```

Writing a good `print` method

A `print` method should match the generic's signature (`x, ...`), do its output via `cat()` or `print()`, and return its input *invisibly* so that `print(x)` can be used in a pipeline without printing twice.

```
1 pct = structure(c(0.12, 0.5, 1), class = "percent")
2
3 print.percent = function(x, ...) {
4   print(paste0(unclass(x) * 100, "%"), quote = FALSE)
5   invisible(x)
6 }
```

```
1 pct
```

```
[1] 12% 50% 100%
```

```
1 z = print(pct)
```

```
[1] 12% 50% 100%
```

```
1 unclass(z)
```

```
[1] 0.12 0.50 1.00
```

Calling `print(x)` (rather than `print(unclass(x))` or `NextMethod()`) inside `print.percent` would dispatch back to

Defining a new generic

A generic is just a function that calls `UseMethod()`. By convention the first argument is the object dispatched on and `...` is included so that methods can add arguments.

```
1 shuffle = function(x, ...) {  
2   UseMethod("shuffle")  
3 }
```

R

```
1 shuffle.default = function(x, ...) {  
2   stop("Class ", class(x), " is not supported by shuffle.", call. = FALSE)  
3 }
```

R

```
1 shuffle.factor = function(x, ...) {  
2   factor( sample(as.character(x)), levels = sample(levels(x)) )  
3 }
```

R

```
1 shuffle.integer = function(x, ...) {  
2   sample(x)  
3 }
```

R

Shuffle results

```
1 shuffle( 1:10 )
```

R

```
[1] 9 7 8 5 2 4 10 3 1 6
```

```
1 shuffle( factor(c("A", "B", "C", "A")) )
```

R

```
[1] B A C A
```

```
Levels: B A C
```

```
1 shuffle( c(1, 2, 3, 4, 5) )
```

R

Error:

! Class numeric is not supported by shuffle.

```
1 shuffle( letters[1:5] )
```

R

Error:

! Class character is not supported by shuffle.

```
1 methods("shuffle")
```

R

```
[1] shuffle.default shuffle.factor shuffle.integer  
see '?methods' for accessing help and source code
```


Implicit classes

Why did `shuffle(c(1, 2, 3, 4, 5))` fail when `class(1)` is `"numeric"`? Objects without a `class` attribute dispatch on an *implicit* class vector that is more detailed than what `class()` reports.

```
1 report = function(x) {  
2   UseMethod("report")  
3 }  
4 report.default = function(x) paste0("Class ", class(x), " does not have a method defined.")  
5 report.integer = function(x) "I'm an integer!"  
6 report.double = function(x) "I'm a double!"  
7 report.numeric = function(x) "I'm a numeric!"
```

```
1 report(1)
```

```
[1] "I'm a double!"
```

```
1 report(1L)
```

```
[1] "I'm an integer!"
```

```
1 report("1")
```

```
[1] "Class character does not have a method defined."
```

```
1 rm(report.integer, report.double)  
2 report(1)
```

```
[1] "I'm a numeric!"
```

```
1 report(1L)
```

```
[1] "I'm a numeric!"
```

```
1 rm(report.numeric)  
2 report(1)
```

```
[1] "Class numeric does not have a method defined."
```

```
1 report(1L)
```

```
[1] "Class integer does not have a method defined."
```

Why?

From [UseMethod](#)'s R documentation:

If the object does not have a class attribute, it has an implicit class. Matrices and arrays have class “matrix” or “array” followed by the class of the underlying vector. Most vectors have class the result of `mode(x)`, except that integer vectors have class `c("integer", "numeric")` and real vectors have class `c("double", "numeric")`.

The implicit class vector can be inspected with `.class2()`,

```
1 .class2(1)
```

R

```
[1] "double" "numeric"
```

```
1 .class2(1L)
```

R

```
[1] "integer" "numeric"
```

```
1 .class2(matrix(1:4, 2))
```

R

```
[1] "matrix" "array" "integer" "numeric"
```

`report(1)` tries `report.double`, then `report.numeric`, then `report.default` - which is why a method for `double` is found

Inheritance

The `class` attribute can be a *vector* - dispatch tries each class in order, so earlier classes “inherit” the methods of later ones. `NextMethod()` calls the method for the next class in the vector.

```
1 x = structure(1:3, class = c("child", "parent"))
2
3 print.parent = function(x, ...) {
4   cat("parent method: ")
5   print(unclass(x))
6   invisible(x)
7 }
```

```
1 print.child = function(x, ...) {
2   cat("child method -> ")
3   NextMethod()
4 }
```

```
1 print(x)
```

child method -> parent method: [1] 1 2 3

`inherits()` tests class membership and is the right tool for validating method arguments,

```
1 inherits(x, "parent")
```

[1] TRUE

```
1 inherits(x, "child")
```

[1] TRUE

```
1 inherits(x, "integer")
```

[1] FALSE

```
1 typeof(x)
```

[1] "integer"

Building an S3 class

Most S3 classes are a list with a class attribute, created by a *constructor* function that validates its inputs. Methods then access the elements with `$` and return *modified copies*.

```
1 rect = function(p1 = c(0, 0), p2 = c(1, 1)) {
2   stopifnot(is.numeric(p1), length(p1) == 2, is.numeric(p2), length(p2) == 2)
3   structure(list(p1 = p1, p2 = p2), class = "rect")
4 }
5
6 area = function(x, ...) UseMethod("area")
7 area.rect = function(x, ...) abs(prod(x$p2 - x$p1))
8
9 set_p1 = function(x, p1, ...) UseMethod("set_p1")
10 set_p1.rect = function(x, p1, ...) {
11   stopifnot(is.numeric(p1), length(p1) == 2)
12   x$p1 = p1
13   x
14 }
15
16 print.rect = function(x, ...) {
17   cat("rect: (", x$p1[1], ",", x$p1[2], ") to (", x$p2[1], ",", x$p2[2], ")\n")
18   invisible(x)
19 }
```

R

Using the class

```
1 r = rect()  
2 r
```

R

```
rect: ( 0 , 0 ) to ( 1 , 1 )
```

```
1 str(unclass(r))
```

R

```
List of 2  
 $ p1: num [1:2] 0 0  
 $ p2: num [1:2] 1 1
```

```
1 area(r)
```

R

```
[1] 1
```

```
1 r2 = set_p1(r, c(-1, -1))  
2 r2
```

R

```
rect: ( -1 , -1 ) to ( 1 , 1 )
```

```
1 r
```

R

```
rect: ( 0 , 0 ) to ( 1 , 1 )
```

Because each method returns the updated object, the class is naturally pipeable,

```
1 rect() |> set_p1(c(-1, -1)) |> area()
```

R

```
[1] 4
```

```
1 area(1:3)
```

R

```
Error in `UseMethod()`:  
! no applicable method for 'area' applied to an
```

```
1 rect(p1 = "a")
```

R

```
Error in `rect()`:  
! is.numeric(p1) is not TRUE
```

Python classes

Basic syntax

A `class` statement bundles *attributes* (data) and *methods* (functions) into a new type. Methods receive the instance as their first argument, conventionally named `self`.

```
1 class rect:
2     """An object representing a rectangle"""
3     p1 = (0, 0)
4     p2 = (1, 2)
5
6     def area(self):
7         return abs((self.p1[0] - self.p2[0]) *
8                     (self.p1[1] - self.p2[1]))
9
10    def set_p1(self, p1):
11        self.p1 = p1
```

```
1 x = rect()
2 x.area()
```

2

```
1 x.p1
```

(0, 0)

```
1 x.set_p1((1, 1))
2 x.area()
```

0

```
1 x.p2 = (3, 3)
2 x.area()
```

4

`x.area()` is shorthand for `rect.area(x)` - Python passes the object as `self` automatically. We have been using this

`__init__`

Calling the class (`rect()`) creates a new *instance* and then calls its `__init__()` method with any arguments - this is the constructor, and where instance attributes should be set.

```
1 class rect:
2     """An object representing a rectangle"""
3
4     def __init__(self, p1=(0, 0), p2=(1, 1)):
5         self.p1 = p1
6         self.p2 = p2
7
8     def area(self):
9         return abs((self.p1[0] - self.p2[0]) *
10                    (self.p1[1] - self.p2[1]))
```

```
1 rect().area()
```

1

```
1 rect((0, 0), (3, 3)).area()
```

9

```
1 z = rect(p1=(-1, -1))
2 z.p1
```

(-1, -1)

```
1 z.p2
```

(1, 1)

Methods whose names start and end with double underscores (`__init__`, `__repr__`, ...) are “dunder” or *special* methods

Class vs instance attributes

Attributes assigned in the class body are *class attributes* shared by every instance (useful for constants), attributes assigned via `self` are *instance attributes* belonging to one object.

```
1 class die:
2     sides = 6
3
4     def __init__(self, value=1):
5         self.value = value
```

```
1 d = die(3)
2 d.value
```

3

```
1 d.sides
```

6

```
1 die.sides
```

6

```
1 die.value
```

AttributeError: type object 'die' has no attribute 'value'

```
1 vars(d)
```

```
{'value': 3}
```

```
1 [m for m in dir(d) if not m.startswith("_")]
```

```
['sides', 'value']
```

`dir()` lists all attributes and methods of an object (including inherited dunder methods), `vars()` returns just the

Mutate or return?

Python methods that mutate an object conventionally modify `self` in place and return `None`. In R, modifying an ordinary vector or list inside a function does not modify the caller's object, so a function must return the updated value. Returning `self` from a Python mutating method instead enables method chaining.

```
1 x = [3, 1, 2]
2 print( x.sort() )
```

py

None

```
1 x
```

py

[1, 2, 3]

```
1 def set_p1(self, p1):
2     self.p1 = p1
3     return self
4
5 rect.set_p1 = set_p1
```

py

```
1 r1 = rect()
2 r2 = r1
3 r1.set_p1((-1, -1)).area()
```

py

4

```
1 r2.p1
```

py

(-1, -1)

```
1 ( rect()
2   .set_p1((-1, -1))
3   .area()
4 )
```

py

4

Methods are just functions stored on the class, so they can be attached after the fact - this keeps the examples short, but in real code define everything inside the `class` block. Note that `r2` was never copied, so it sees the change made through

__str__ and __repr__

Every object has a default string representation, but it is not very useful,

```
1 rect()
```

py

```
<__main__.rect object at 0x10b146bd0>
```

```
1 print(rect())
```

py

```
<__main__.rect object at 0x10b0cc3b0>
```

`__repr__()` should return an unambiguous representation, ideally valid Python that recreates the object. `__str__()` is used by `print()` and `str()`, and falls back to `__repr__()` if not defined.

```
1 def rect_repr(self):  
2     return f"rect({self.p1}, {self.p2})"  
3  
4 rect.__repr__ = rect_repr
```

py

```
1 rect()
```

py

```
rect((0, 0), (1, 1))
```

```
1 print(rect())
```

py

```
rect((0, 0), (1, 1))
```

```
1 [rect(), rect((1, 1), (2, 2))]
```

py

```
[rect((0, 0), (1, 1)), rect((1, 1), (2, 2))]
```

Other special methods

Operators and built in functions also dispatch to special methods - this is the mechanism behind the operator overloading we saw in Lecture 2 ("abc" * 3, [1] + [2]).

```
1 def rect_eq(self, other):  
2     return self.p1 == other.p1 and self.p2 ==  
3  
4 def rect_contains(self, pt):  
5     return all(min(a, b) <= x <= max(a, b)  
6                 for a, b, x in zip(self.p1, se  
7  
8 rect.__eq__ = rect_eq  
9 rect.__contains__ = rect_contains
```

```
1 rect() == rect()
```

True

```
1 rect() == rect((1, 1), (2, 2))
```

False

```
1 (0.5, 0.5) in rect()
```

True

```
1 (2, 2) in rect()
```

False

Without `__eq__`, `==` compares object identity (like `is`). See the [data model docs](#) for the full list, including `__len__`.

Inheritance

A class can *inherit* from another, gaining all of its attributes and methods. The subclass can add new methods and override existing ones - `super()` gives access to the parent's version.

```
1 class square(rect):  
2     def __init__(self, p1=(0, 0), l=1):  
3         if not isinstance(l, (int, float)):  
4             raise TypeError("l must be a number")  
5         self.l = l  
6         super().__init__(p1, (p1[0] + l, p1[1]  
7  
8     def __repr__(self):  
9         return f"square({self.p1}, {self.l})"
```

```
1 s = square((1, 1), 2)  
2 s
```

```
square((1, 1), 2)
```

```
1 s.p2
```

```
(3, 3)
```

```
1 s.area()
```

```
4
```

```
1 (2, 2) in s
```

```
True
```

```
1 square(l="a")
```

```
TypeError: l must be a number
```

This is the same idea as R's class vector `c("square", "rect")`, with `super()` playing the role of `NextMethod()`. Python's

isinstance() vs type()

`isinstance()` respects inheritance while `type()` reports the exact class - the former is almost always what you want when validating inputs, but be aware of what a type inherits from.

```
1 isinstance(s, square)
```

py

True

```
1 isinstance(s, rect)
```

py

True

```
1 type(s) is rect
```

py

False

```
1 isinstance(3, (int, float))
```

py

True

```
1 isinstance("3", (int, float))
```

py

False

```
1 isinstance(True, int)
```

py

True

```
1 type(True) is int
```

py

False

```
1 bool.__mro__
```

py

(<class 'bool'>, <class 'int'>, <class 'object'>)

`__mro__` (method resolution order) is the tuple of classes Python searches for inherited attributes and methods - roughly analogous to the dispatch order in an R S3 class vector. Since `bool` inherits from `int`, code accepting `ints` accepts `True`

Exercise 2

Implement a `circle` class in Python and the equivalent S3 class in R.

Python:

- `__init__(self, r=1)` stores the radius and raises a `ValueError` if it is negative.
- `area()` returns πr^2 .
- `__repr__()` returns `circle(r=1)`.

R:

- `circle(r = 1)` validates `r` and returns a list with class `"circle"`.
- an `area()` method for the `area` generic defined earlier.
- a `print.circle()` method that returns its input invisibly.

Check that `circle(2)`, `area(circle(2)) / circle(2).area()`, and `circle(-1)` behave as expected.

Comparing R & Python

Object systems summary

Concept	R (S3)	Python
define a class	<code>structure(list(...), class = "x")</code>	<code>class x: with __init__()</code>
where methods live	separate functions <code>generic.class()</code>	inside the class definition
dispatch	<code>UseMethod()</code> on the first argument's class	<code>obj.method()</code> on the object's type
printing	<code>print.x()</code> (returns <code>invisible(x)</code>)	<code>__repr__()</code> and <code>__str__()</code>
operators	<code>Ops.x(), "+.x"()</code>	<code>__add__(), __eq__(), ...</code>
inheritance	<code>class = c("child", "parent")</code>	<code>class child(parent):</code>
call the parent method	<code>NextMethod()</code>	<code>super().method()</code>
type check	<code>inherits(x, "class")</code>	<code>isinstance(x, cls)</code>
list methods	<code>methods(class = "x")</code>	<code>dir(x)</code>

Functional vs in-place

The deeper difference is about *state*. Ordinary R vectors and lists behave like values: modifying a function argument does not modify the caller's object, so updated values are normally returned and reassigned. This naturally supports functional pipelines such as `game |> guess("word") |> guess("other")`.

Python variables refer to objects. A mutating method can change an object visible through every reference to it; by convention, many such methods return `None`, although returning `self` can support chaining.

```
1 r = rect()
2 set_p1(r, c(-1, -1))
```

R

rect: (-1 , -1) to (1 , 1)

```
1 r$p1
```

R

```
[1] 0 0
```

```
1 r = rect()
2 r.set_p1((-1, -1))
```

py

rect((-1, -1), (1, 1))

```
1 r.p1
```

py

```
(-1, -1)
```

Neither is better - but mixing the conventions (an R method that silently modifies a global, or a Python method that

Takeaways

- R lists are heterogeneous vectors - `[]` returns a list, `[[` and `$` return a single element, and assigning `NULL` removes one.
- Attributes are metadata attached to any R object, `structure()` sets them in one call, and `class` is the attribute S3 dispatches on.
- An S3 generic is a function that calls `UseMethod()`, methods are just functions named `generic.class`, and `default` catches everything else.
- Objects without a `class` attribute dispatch on an implicit class such as `c("double", "numeric")`.
- Python classes bundle data and methods, `__init__()` constructs, `__repr__()` / `__str__()` display, and `isinstance()` checks type with inheritance.
- R functions commonly return updated values; Python mutating methods commonly update objects in place - know which convention an interface follows.